

VUE.JS

Aula 6: Consumindo APIs

Buscando dados de verdade com requisições assíncronas

O que vamos ver hoje

1

Por que buscar dados de uma API?

Dados fixos no código x dados de verdade

2

fetch() e Promises

Fazendo a primeira requisição

3

async/await

Escrevendo código assíncrono de forma mais limpa

4

Carregando dados e tratando erros

created(), loading e try/catch

5

Prática

Lista de tarefas vinda de uma API real

AULA 6 / 7

Hoje sua aplicação para de inventar dados e passa a buscar informações de verdade, vindas de um servidor.

Revisão rápida da Aula 5

Antes de seguir, lembre o que já sabemos fazer:

routes

Lista que liga cada path a um componente

router-link

Cria links que trocam de rota

router-view

Onde o componente da rota aparece

\$router.push()

Navega para outra rota via código

Hoje: até agora todo dado veio de dentro do próprio data(). Vamos aprender a buscar dados reais em um servidor.

Por que buscar dados de uma API?

Até agora, todo dado das nossas aulas nascia fixo dentro do `data()`. Isso tem um limite.

Dados fixos no código

- ✗ A lista de tarefas é sempre a mesma toda vez que a página abre
- ✗ Para mudar um dado, é preciso editar o código e publicar de novo
- ✗ Cada pessoa que abre o site vê os mesmos dados de mentira
- ✗ Não reflete o que está guardado de verdade em um banco de dados

Dados vindos de uma API

- ✓ Os dados vêm de um servidor e podem mudar sem tocar no código
- ✓ Cada usuário pode ver seus próprios dados reais
- ✓ O mesmo componente Vue funciona para qualquer dado que a API devolver
- ✓ Abre caminho para também enviar dados de volta (salvar, editar, excluir)

API (Application Programming Interface) aqui significa: um servidor que responde com dados quando fazemos uma requisição HTTP.

fetch(): fazendo uma requisição

fetch() é uma função nativa do JavaScript (não é do Vue) para pedir dados a um endereço da web.

```
fetch("https://api.exemplo.com/tarefas")
  .then((resposta) => resposta.json())
  .then((dados) => {
    console.log(dados);
  })
  .catch((erro) => {
    console.log("deu erro:", erro);
  });
```

Repare

fetch(url) começa a requisição e devolve uma Promise (próximo slide)

resposta.json() transforma a resposta em um objeto/array do JavaScript — também é assíncrono

.catch() captura falhas de rede (sem internet, servidor fora do ar)

Assíncrono quer dizer: o Vue não trava esperando a resposta. O código continua rodando, e o .then() só executa quando a resposta chegar.

Promises: o que fetch() devolve

Uma Promise é uma "promessa" de um valor que ainda vai chegar. Ela sempre termina em um de dois estados.



Você não escreve código para "esperar" — **você entrega funções** (`.then` / `.catch`) que o JavaScript chama sozinho quando a Promise chegar em `resolved` ou `rejected`.

Toda vez que uma função é assíncrona (`fetch`, `.json()`, leitura de arquivo...), ela devolve uma Promise.

async/await: a mesma coisa, mais legível

async/await é uma forma alternativa de escrever Promises — mesmo comportamento, código mais parecido com o síncrono.

Com .then()

```
buscarTarefas() {  
  fetch("/api/tarefas")  
    .then((r) => r.json())  
    .then((dados) => {  
      this.tarefas = dados;  
    })  
    .catch((e) => console.log(e));  
}
```

Com async/await

```
async buscarTarefas() {  
  const resposta = await fetch(  
    "/api/tarefas"  
  );  
  const dados = await resposta.json();  
  this.tarefas = dados;  
}
```

async marca que a função tem código assíncrono dentro. **await** "pausa" a função até a Promise resolver, sem travar o restante da aplicação — e o resultado sai direto na variável, sem .then().

Buscando dados ao carregar o componente

O lugar mais comum para buscar dados é no `created()` — assim que os dados existem, a busca já começa.

```
export default {
  data() {
    return {
      tarefas: [],
      carregando: true
    };
  },
  async created() {
    const resposta = await fetch(
      "/api/tarefas"
    );
    this.tarefas = await resposta.json();
    this.carregando = false;
  }
};
```

Por que carregando?

A requisição leva um tempo (mesmo que curto) para responder. Enquanto isso, a tela precisa mostrar algo — não pode ficar em branco ou quebrada.

O padrão é começar com `carregando: true`, mostrar um aviso de "carregando..." no template, e virar `false` assim que os dados chegam.

Tratando erros com try/catch

Requisições podem falhar (sem internet, servidor fora do ar). try/catch evita que isso quebre a aplicação.

```
async created() {  
  try {  
    const resposta = await fetch(  
      "/api/tarefas"  
    );  
    this.tarefas = await resposta.json();  
  } catch (erro) {  
    this.erro = "Não deu para carregar";  
    console.log(erro);  
  } finally {  
    this.carregando = false;  
  }  
}
```

try

O código que pode falhar

catch (erro)

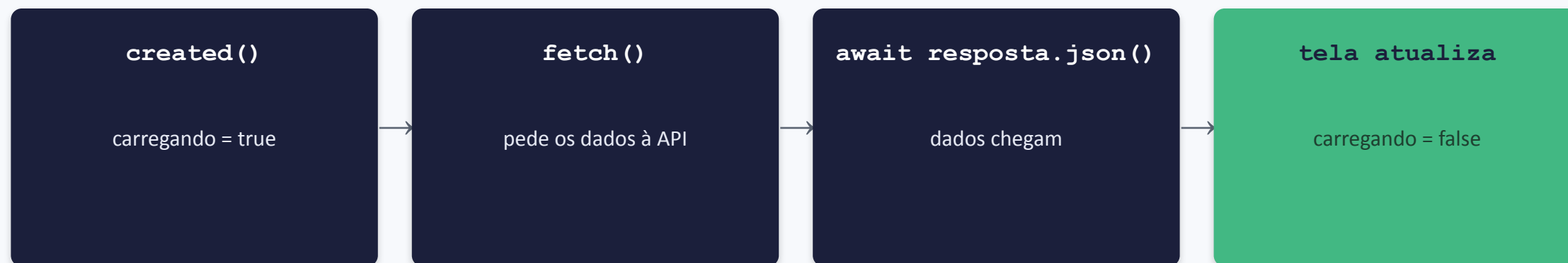
Roda só se algo dentro do try falhar — aqui é onde avisamos o usuário

finally

Roda sempre, deu certo ou não — ótimo lugar para desligar o carregando

O ciclo completo de uma requisição

Do componente sendo criado até a tela mostrar os dados de verdade.



Enquanto `carregando` é `true`, o template mostra um aviso de carregamento. Assim que os dados chegam (com ou sem erro), a tela re-renderiza sozinha — é a mesma reatividade da Aula 1.

EXERCÍCIO PRÁTICO

Lista de tarefas vinda de uma API real

O que construir

- Na tela Tarefas.vue da Aula 5, busque dados de <https://jsonplaceholder.typicode.com/todos>
- Use data() com tarefas: [], carregando: true e erro: null
- Busque os dados em um created() assíncrono, com try/catch/finally
- Enquanto carregando, mostre "Carregando tarefas..."
- Se erro, mostre uma mensagem amigável
- Exiba as 10 primeiras tarefas com v-for

```
<template>
  <p v-if="carregando">Carregando...</p>
  <p v-else-if="erro">{{ erro }}</p>
  <ul v-else>
    <li v-for="t in tarefas.slice(0, 10)"
      :key="t.id">
      {{ t.title }}
    </li>
  </ul>
</template>
```

Recapitulando

`fetch()`

Faz uma requisição HTTP e devolve uma Promise

`Promise`

Um valor que ainda vai chegar — resolved ou rejected

`async/await`

Escreve código assíncrono no estilo síncrono

`try/catch/finally`

Trata erros sem quebrar a aplicação

`carregando / erro`

Estados que guiam o que o template mostra

Próxima aula: Pinia — gerenciamento de estado compartilhado, e projeto final